

CS 236: Operating Systems Lab

Spring 2025-26

Lab 5: show me the memory!

xv6 is a teaching operating system (developed at MIT) — an OS meant for teaching design and implementation concepts of an OS.

To install xv6 on Ubuntu, look up the details described in **Appendix A**.

The Holy Grail for xv6

- [xv6-risc book](#)
- [xv6-risc src booklet](#)

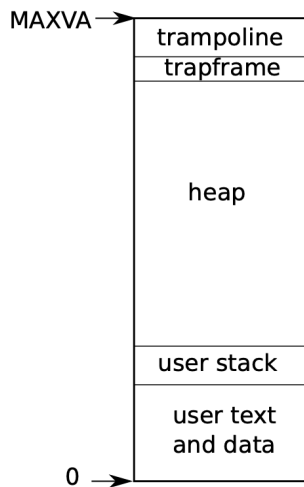
The xv6 source tree is organized to mirror OS subsystems:

- **kernel/** → process management, memory, file system, traps
- **user/** → user programs
- **mkfs/** → file system creation tool

Each directory corresponds to a logical OS component, enabling systematic exploration and modification. **You will mostly interact with the **kernel/** and **user/** directory.**

1. yes! there is a virtual world out there

In xv6, every process operates within its own virtual address space. As illustrated in the figure below, a process's user memory starts at virtual address zero. The layout follows a specific order: instructions (text) come first, followed by global variables, the stack, and finally the "heap" area, which the process can expand dynamically as needed.



However, even though the virtual address space appears contiguous and spans from 0 to MAXVA, not all of these addresses are actually mapped to physical memory (RAM).

1. a: validating virtual addresses

For this task, implement a system call that determines whether a specific virtual address is valid and backed by physical memory.

```
int pte_valid(uint64 va);
```

This function should take a virtual address (va) as an argument. It must return 1 if the address maps to a valid physical page and 0 if it does not.

Hints:

- Review `kernel/vm.c` carefully. Specifically, look at the logic in `ismapped()` and `walk()` to understand it's working.

Testing and Sample Output:

We have provided a sample program, `p1a.c`, which uses this system call. You can use this to verify your implementation.

```
Shell
$ p1a
VA: 0x00000000000003FBC valid: 1
VA: 0x0000000100000000 valid: 0
```

1. b: matter of flags

Just as a virtual address can be mapped (valid) or unmapped (invalid), there are other permission information items (bits in the pte flag) that govern memory access.

For this task, implement a system call that inspects and prints the permission flags for a given virtual address.

void get_pteflags(uint64 va);

This function takes a virtual address (**va**) as an argument. It should look up the corresponding page table entry and print whether the address is:

- Readable (R)
- Writable (W)
- Executable (X)
- User Accessible (U)

Hints:

- Check the **ismapped()** logic in **kernel/vm.c**, particularly how page table entries are checked with different flags.
- Refer to **kernel/riscv.h** to find the bitmask definitions for these flags (e.g., **PTE_R**, **PTE_W**, etc.).

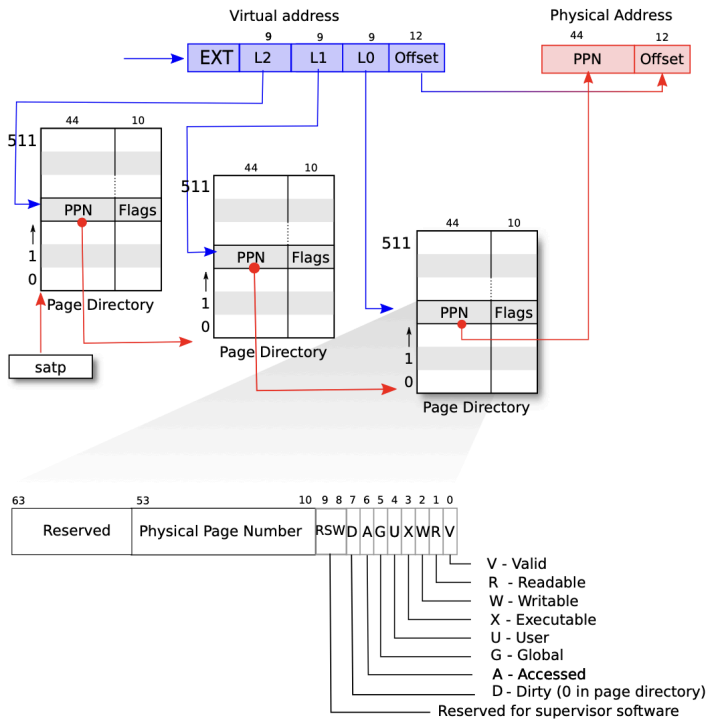
Testing and Sample Output:

We have provided a sample program, **p1b.c**, which uses this system call. You can use this to verify your implementation.

```
Shell
$ p1b
VA: 0x0000000000000000 -> R:1 W:0 X:1 U:1
VA: 0x0000000000003fac -> R:1 W:1 X:0 U:1
VA: 0x0000000000004000 -> R:1 W:1 X:0 U:1
VA: 0x0000000000001000 -> R:1 W:1 X:0 U:1
VA: 0x0000000000009000 -> R:1 W:0 X:1 U:1
VA: 0x0000003fffffff000 -> R:1 W:0 X:1 U:0
```

2. is any of this real? (or is it all virtual?)

Page tables are the mechanism used by the hardware to map virtual addresses to physical addresses. Since user-space programs use virtual addresses, the CPU must translate them into physical addresses in RAM to perform actual memory accesses. As illustrated in the figure below, the physical address of the root page table is stored in the `satp` register. Which is then automatically used by the MMU for translations.



xv6 utilizes two distinct types of page tables —

- Kernel Page Table: A single global page table used by the kernel in supervisor mode.

- User Page Tables: Each process has its own private page table used when executing in user mode.

2. a: where are my pagetables?

To understand the separation between user and kernel views of memory, implement a system call that reveals the physical locations of these page tables.

```
void print_pgdirs(void);
```

This function should print three values to the console:

- The physical address of the root Kernel Page Table directory.
- The physical address of the root User Page Table directory for the current process.
- The current physical address is stored in the `satp` register.

Hints:

- Look at `kernel/proc.h` to find the user pagetable details.
- Look at `kernel/vm.c` to find the global variable used for the kernel page table.
- To read the satp register, search for helper functions in `kernel/riscv.h` (look for `r_satp()`)
- Keep in mind that the satp register contains more than just the pointer to the page directory. You can look at `MAKE_SATP` to see how the physical address is packed into the register. You will need to reverse this process to retrieve the address.

Testing and Sample Output:

We have provided a sample program, `p2a.c`. Running this should display the addresses.

```
Shell
$ p2a
Physical Address of Kernel root pagetable : 0x0000000087fff000
Physical Address of User root pagetable   : 0x0000000087f26000
Physical Address of Current satp register : 0x0000000087fff000
```

2. b: is there a route from virtual to physical?

Now that we know where the page tables are located, let's perform the translation. Implement two system calls, one that translates a virtual address to its corresponding PTE and another that translates it to its physical address.

```
uint64 va_to_pte(uint64 va);
uint64 va_to_pa(uint64 va);
```

These functions should take a virtual address (`va`) and return the physical address (`pa`) or pte that it maps to. If the virtual address is not mapped, return `-1`.

Hints:

- Review `walk()` and `walkaddr()` in `kernel/vm.c`. These functions implement the software page table walk and can be used or mimicked to find the physical address.

Use the provided **p2b.c** to test your system call with various addresses.

```
Shell
$ p2b
User VA : 0x0000000000004000
User PTE : 0x0000000087F23000
User PA : 0x0000000087F23000
---
User VA : 0x0000000000004001
User PTE : 0x0000000087F23000
User PA : 0x0000000087F23001
---
User VA : 0x0000000000005000
User PTE : 0x0000000087F48000
User PA : 0x0000000087F48000
```

Note that you can rerun this with memory accesses on the same page and on different pages! What are the changes in output?

2. c: we are the same, or are we?

Let's investigate how **fork()** affects memory mappings. We know that **fork()** creates a new process by copying the parent's memory image, meaning the virtual address space is replicated. But does that mean the physical addresses are the same?

Task: Create a user program **p2c.c**.

1. Allocate a variable or use a fixed virtual address.
2. Call **fork()** to create a child process.
3. In both the parent and the child, print the **virtual address** of the variable and its **physical address** (using your **va_to_pa** system call).

Compare the outputs. Are the virtual addresses the same? Are the physical addresses the same? Explain your observations in **reports/p2c.txt**

3. laziness pays?

xv6 allows us to dynamically change a process's heap size at runtime using the `sbrk()` system call. This allocation can be performed in two modes: **Eager Allocation**, where physical pages are allocated and mapped immediately when the process size increases; or **Lazy Allocation**, where the kernel simply increases the process size without actually allocating physical RAM.

If memory is allocated lazily, accessing a newly allocated virtual address will trigger a page fault. The kernel's trap handler catches this fault, allocates the physical page, and resumes execution.

3. a: the address space quantum

For the first task, implement a system call to retrieve the current size of the process's virtual address space.

`uint64 getvasize(void);`

This function should return the total size of the virtual address space (in bytes) currently allocated to the process.

Hints:

- You should examine the `struct proc` definition in `kernel/proc.h`. Specifically, look for the field that tracks the process size.
- You should also review `sys_sbrk` in `kernel/sysproc.c` to understand how the virtual size changes during allocation.

Testing and Sample Output:

Use the provided `p3a.c` to verify that the size increases as expected when you call `sbrk`.

```
Shell
$ p3a
-----SBRK LAZY:-----
Virtual memory size before sbrk lazy : 16384 bytes
Allocating 4096 bytes lazily...
Virtual memory size after sbrk lazy: 20480 bytes

-----SBRK EAGER:-----
Virtual memory size before sbrk eager: 20480 bytes
Allocating 4096 bytes eagerly...
Virtual memory size after sbrk eager: 24576 bytes
```

3.b: the *real* address space quantum

Now we need to distinguish between the virtual size and the physical resources actually consumed. Implement a system call that returns the amount of physical memory in use.

```
uint64 getpasize(void);
```

This function must return the total amount of **Physical RAM** (in bytes) currently mapped to the process. Unlike `getvasize`, this requires you to iterate through the process's page table, count the number of valid pages, and multiply that count by the page size (`PGSIZE`).

Hints:

- You need to use `walk()` to walk your va space.
- You can use the `ismapped()` method to check whether physical pages are mapped.

Testing and Sample Output:

Use `p3b.c` to check if `getvasize` returns a large number while `getpasize` remains small.

```
Shell
$ p3b
-----SBRK LAZY:-----
Physical memory size before: 16384 bytes
Virtual memory size before: 16384 bytes

Allocating 4096 bytes lazily...
Physical memory size after: 16384 bytes
Virtual memory size after: 20480 bytes

-----SBRK EAGER:-----
Physical memory size before: 16384 bytes
Virtual memory size before: 20480 bytes

Allocating 4096 bytes eagerly...
Physical memory size after: 20480 bytes
Virtual memory size after: 24576 bytes
```


3.c: limit va → pa

Modify the kernel to maintain a count of page faults for each process. Then, implement a system call to expose this counter to the user — return the number of page faults that have occurred for the calling process since its creation.

```
int getlazyfaults(void);
```

Hints:

- Maintain a page fault count using `struct proc` in `kernel/proc.h`.
- Look at the trap handler in `kernel/trap.c` (specifically `vmfault` in `usertrap`).

Use `p3c.c`. This program will allocate a large chunk of memory and access it page by page. You should see the page fault count increase as the program touches new pages.

```
Shell
$ p3c
Page faults Before sbrk: 0
Virtual address space size before sbrk lazy: 16384 bytes

Allocating 5 pages lazily where each page is of size 4096 bytes...
Virtual address space size after sbrk lazy: 36864 bytes

Touched page at virtual address: 0x0000000000004000
Page faults after touching page 0: 1
PA size after each page touch: 20480 bytes

Touched page at virtual address: 0x0000000000005000
Page faults after touching page 1: 2
PA size after each page touch: 24576 bytes

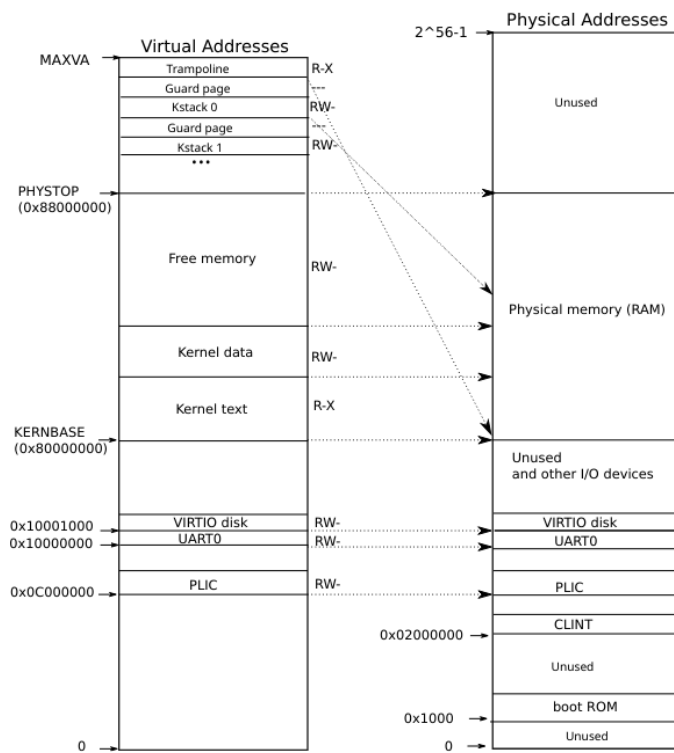
Touched page at virtual address: 0x0000000000006000
Page faults after touching page 2: 3
PA size after each page touch: 28672 bytes

Touched page at virtual address: 0x0000000000007000
Page faults after touching page 3: 4
PA size after each page touch: 32768 bytes

Touched page at virtual address: 0x0000000000008000
Page faults after touching page 4: 5
PA size after each page touch: 36864 bytes

Total Page faults After touching pages: 5
```

4. kernel. hence, the user.



The kernel virtual address space is distinctly different from the user address space. As illustrated in the figure, the virtual address range from **KERNBASE** to **PHYSTOP** contains the kernel text, data, and the free memory that the kernel uses for its internal operations.

Addresses below **KERNBASE** are reserved for interacting with hardware devices (which will be explored in future labs). Interestingly, at the very top of the address space, we find the **TRAMPOLINE** page and the Kernel Stacks for every process. These are pre-allocated for the total number of supported processes when the operating system boots. **Guard pages** are placed between these stacks to prevent a stack overflow in one process from corrupting the stack of another.

4. a: jumping on the TRAMPOLINE!

In xv6, the "Trampoline" is a special page mapped at the very top of the virtual address space of every user space program and the kernel, specifically from **MAXVA - PGSIZE** to **MAXVA**. This page is critical for switching between user and kernel modes.

Task: Create a user program **p4a.c**.

Pass the trampoline address (**MAXVA - PGSIZE**) to your **va_to_pa** system call. Print the result and note it.

Testing and Sample Output:

```
Shell
$ p4a
Trampoline virtual address : 0x0000003FFFFFF000
Trampoline physical address (va_to_pa): 0x0000000080006000
```

4. b: secrets of the kernel page table!

Now that we have looked at how the **TRAMPOLINE** page is mapped in userspace, let's try to understand what is happening inside the kernel virtual address space.

Implement a new system call that takes a kernel virtual address and prints its mapped physical address.

```
uint64 kva_to_pa(uint64 va);
```

This function must perform the translation using the **Kernel Page Table** instead of the current process's user page table.

Use the provided program **p4b.c** to print the virtual and physical addresses of: The first 10 pages starting from **KERNBASE**, and the top 10 pages starting from **MAXVA**

```
Shell
$ p4b
VA TO PA for 10 pages starting from KERNBASE:
Kernel virtual address => physical address
0x0000000080000000 => 0x0000000080000000
0x0000000080001000 => 0x0000000080001000
0x0000000080002000 => 0x0000000080002000
0x0000000080003000 => 0x0000000080003000
0x0000000080004000 => 0x0000000080004000
0x0000000080005000 => 0x0000000080005000
0x0000000080006000 => 0x0000000080006000
0x0000000080007000 => 0x0000000080007000
0x0000000080008000 => 0x0000000080008000
0x0000000080009000 => 0x0000000080009000
```

```
VA TO PA for 10 pages from MAXVA
Kernel virtual address => physical address
0x0000003FFFFFFF000 => 0x0000000080006000
0x0000003FFFFFFE000 => Invalid address
0x0000003FFFFFD000 => 0x0000000087F99000
0x0000003FFFFFC000 => Invalid address
0x0000003FFFFFB000 => 0x0000000087F98000
0x0000003FFFFFA000 => Invalid address
0x0000003FFFFF9000 => 0x0000000087F97000
0x0000003FFFFF8000 => Invalid address
0x0000003FFFFF7000 => 0x0000000087F96000
0x0000003FFFFF6000 => Invalid address
```

What did you observe in the output? Given that the top pages from **MAXVA** contain the TRAMPOLINE and Kernel Stacks of processes, what happened when you tried to translate them using the kernel page table? Explain your observations in **reports/p4b.txt**

4. c: system calls on steroids

Some operating systems (e.g., Linux) speed up certain system calls by sharing data in a read-only region between userspace and the kernel. This eliminates the need for kernel crossings when performing these system calls.

Your objective is to implement a mechanism that allows the kernel to share the process ID (PID) directly with the user process via a shared memory page.

You must modify the kernel to allocate a physical page for each process, store the process's PID in that page, and map it into the process's virtual address space at a specific location (at the next page after **TRAPFRAME**).

On the user side, you should create a new function **ugetpid()** that reads the PID directly from this virtual address, bypassing the standard system call mechanism.

Hints:

- Look into **kernel/proc.c**, specifically the **proc_pagetable** function.
- Check how the **TRAMPOLINE** and **TRAPFRAME** are mapped in the kernel.
- Add proper flags that ensure that the userspace can only read and not write or execute using the mapped address.
- You also need to free the page when the process is destroyed.

Testing and Sample Output:

Use the provided program **p4c.c** to verify if your implementation is really faster. The program will compare the result of the standard **getpid()** system call with your new **ugetpid()** function to ensure they return the same value.

```
Shell
$ p4c
PID from getpid() : 3 time-taken: 15
PID from ugetpid(): 3 time-taken: 0
```

submission guidelines

Create a patch with:

```
Shell  
➤ ./patch.sh
```

Rename the generated **lab5.patch** to **<rollno>_lab5.patch** and submit it to Moodle

Appendix A

Follow these steps for installing xv6 on a Ubuntu-based Linux machine:

1. Prerequisites
 - a. First, update your system:
sudo apt update
 - b. Install the required packages:
sudo apt install -y git build-essential qemu-system-riscv64 gcc-riscv64-unknown-elf
 - **build-essential**
For essential packages like libc, gcc, make, etc.
 - **qemu-system-riscv64**
Emulates a RISC-V machine in software.
Your computer's real CPU speaks x86-64, but xv6 is written for RISC-V.
QEMU **pretends to be a RISC-V computer**, so the OS can run even though the hardware is different.
 - **gcc-riscv64-unknown-elf**
It runs on your x86-64 machine but produces **RISC-V binaries**, not x86-64 ones.
We need this because xv6 must be compiled for the **CPU architecture that QEMU is emulating**.
2. Verify toolchain installation
riscv64-unknown-elf-gcc --version
qemu-system-riscv64 --version
If both commands work, your environment is ready.
3. Clone the xv6 RISC-V Repository
git clone <https://github.com/mit-pdos/xv6-riscv.git>
cd xv6-riscv
4. Build xv6
make
Run the command to build the OS. If compilation succeeds, you are ready to run xv6.
5. Run xv6 on QEMU
Start the xv6 operating system:
make qemu

Appendix B

A list of riscv ISA items and xv6 items that intersect with the xv6 system call implementation — For details of these and more, look up Chapter 4 of the xv6 book.

C/C++

ecall

sret

stvec

sepc

scause

sstatus (SIE and SPP)

sscratch

satp

csrw

a0

TRAMPOLINE

TRAPFRAME

uservec

usertrap

userret

prepare_return

syscall

copyin

copyout

copyinstr

argint

argstr